Wrapper Classes in Java

01 November 2026 10:53

Library classes Introduction to wrapper classes, methods of wrapper class and their usage with respect to numeric and character data types. Autoboxing and Unboxing in wrapper classes. Class as a composite type, distinction between primitive data type and composite data type or class types. Class may be considered as a new data type created by the user, that has its own functionality. The distinction between primitive and composite types should be discussed through examples. Show how classes allow user defined types in programs. All primitive types have corresponding class wrappers. Introduce Autoboxing and Unboxing with their definition and simple examples.

The following methods are to be covered:
int parseInt(String s), long parseLong(String s), float parseFloat(String s), double parseDouble(String s), boolean isDigit(char ch), boolean isLetter(char ch), boolean isLetterOrDigit(char ch), boolean isLowerCase(char ch), boolean isUpperCase(char ch), boolean isWhitespace(char ch), char toLowerCase (char ch) char toUpperCase(char ch)

Boxing, Autoboxing and Unboxing in Java
In Java, boxing, auto boxing and unboxing are concept that explain the conversion between primitive data types and their corresponding wrapper classes. Java provides. These features to make it easier to work with primitive values as objects.

Understanding primitive data types and a wrapper classes
Before understanding boxing and unboxing, we must understand primitive data types and there is corresponding wrapper classes.

Java has eight primitive data types. Each primitive data type has a corresponding wrapper class available in java.lang package.

Primitive Data Type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

```
int x = 100; ⟵ primitive type
x
100   (int)
Integer i = 200; ⟵ Integer is a wrapper class.
i                  The Integer class wraps an integer value
<object reference>   (Integer)   inside an object.
System.out.print(i);             This is called boxing.
i = x;
i
```

```
<object reference>   (Integer)
System.out.print("\n"+i);
```

What is Boxing in Java?
Boxing is the process of converting a primitive data type into its corresponding wrapper class object.
In simple words, converting a primitive value into an object is called **Boxing.**

The main purpose of Boxing is we get multiple methods to manipulate primitive data.

What is Autoboxing in Java?
Auto boxing is the automatic conversion of a primitive data type into its corresponding wrapper class object by the Java compiler.
In simple words, when Java automatically converts a primitive value into an object, it is called auto boxing.

```
Integer x = 100;
```

What is unboxing in Java?
Unboxing is the process of converting a wrapper class object into its corresponding primitive data type.

In simple words, converting an object into a primitive value is called unboxing.

```
Integer x = 100; //boxing
int y = x; //Unboxing
```

What is automatic unboxing in Java?
Automatic unboxing is the automatic conversion of wrapper class object into its corresponding primitive data type by the Java compiler.

Difference between boxing, auto boxing and unboxing?

Concept	Meaning	Conversion
Boxing	Explicit conversion of primitive into object.	int->Integer
Autoboxing	Automatic conversion of primitive into object.	int->Integer
Unboxing	Explicit conversion of object into primitive.	Integer->int
Automatic unboxing	Automatic conversion of object into primitive.	Integer->int

Remember these statement:

```
//Boxing
Integer obj1 = Integer.valueOf(100);

//Autoboxing
Integer obj2 = 100;

//Unboxing
```

```
int a = obj1.intValue();

//Automatic Unboxing
int b = obj2;
```

Wrapper Classes in Java

In Java, Rapper classes are special classes that convert primitive data type into objects. Java provides eight primitive data types:

- byte
- short
- int
- long
- float
- double
- char
- boolean

Each primitive data type has a corresponding wrapper class available in the java.lang package. A wrapper class wraps up primitive value inside an object.

For example:

- int is wrapped by Integer.
- char is wrapped by Character.
- double is wrapped by Double.
- Float is wrapped by Float.

Integer Wrapper Class

The **Integer** class is a wrapper class for the primitive **int** data type.

It provides several useful methods for:

- Converting strings into integer.
- Converting integers into string.
- Comparing two integer values.
- Finding the minimum and maximum integer values.
- Converting decimal numbers into binary, octal, and hexadecimal representations.
- Retrieving integer values from wrapper objects.

Important methods of Integer class

Method	Description
Integer.valueOf()	Returns an Integer object.
Integer.parseInt()	Converts a string into primitive int.
intValue()	Converts Integer object into int.
toString()	Converts an integer into a string.
compare()	Compare two integer values.
max()	Returns the maximum of two integers.
min()	Returns the minimum of two integers.
sum()	Adds two integer values.
toBinaryString()	Converts an integer into binary.

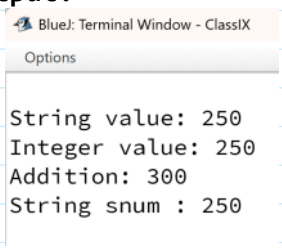
toOctalString()	Converts an integer into octal.
toHexString()	Converts an integer into hexadecimal.
bitCount()	Count the number of one bits.
reverse()	Reverses the bits of an integer.

```
package BoxingAndUnBoxing;


/**
 * Write a description of class IntegerParseDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class IntegerParseDemo
{
    public static void main(String[] args){
        System.out.print("\f");
        String str = "250";
        int num = Integer.parseInt(str);
        System.out.print("\nString value: " + str);
        System.out.print("\nInteger value: " + num);
        System.out.print("\nAddition: " + (num + 50));

        String snum = Integer.toString(num);
        System.out.print("\nString snum : " + snum);
    }
}
```

Output:




```
package BoxingAndUnBoxing;


/**
 * Write a description of class IntegerValueOfDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class IntegerValueOfDemo
{
    public static void main(String[] args){
        String str = "500";
        Integer iob = Integer.valueOf(str);
        System.out.print("\nInteger object : " + iob);

        int result = iob + 100;
```

```
        System.out.print("\nResult: " + result);
    }
}
```

Output:

```
String snum : 250
Integer object : 500
Result: 600
```

```
package BoxingAndUnBoxing;


/**
 * Write a description of class IntegerMaxMinDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class IntegerMaxMinDemo
{
    public static void main(String[] args){
        int a = 75;
        int b = 75;

        System.out.print("\fFirst Number: " + a);
        System.out.print("\nSecond Number: " + b);

        System.out.print("\nMaximum: " + Integer.max(a, b));
        System.out.print("\nMinimum: " + Integer.min(a, b));
        System.out.print("\nMaximum: " + Integer.max(a, 35));
        System.out.print("\nMinimum: " + Integer.min(a, 35));
    }
}
```

Output:

```
BlueJ: Terminal Window - ClassIX
Options
First Number: 75
Second Number: 75
Maximum: 75
Minimum: 75
Maximum: 75
Minimum: 35
```

Comparing two integer values

The compare () method compares to integer values.
It returns:

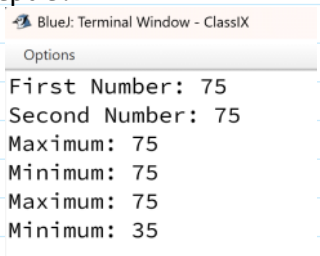
- Negative value if the first integer is smaller.
- 0 if both integers are equal.
- Positive value if the first integer is greater.

```
package BoxingAndUnBoxing;
```

```
/**
 * Write a description of class IntegerCompareDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class IntegerCompareDemo
{
    public static void main(String[] args){
        int a = 100;
        int b = 200;

        int result = Integer.compare(a,b);
        System.out.print("\fResult: " + result);
        if(result > 0){
            System.out.print("\nFirst number is greater.");
        }
        else if (result < 0){
            System.out.print("\nSecond number is greater.");
        }
        else{
            System.out.print("\nBoth numbers are equal.");
        }
    }
}
```

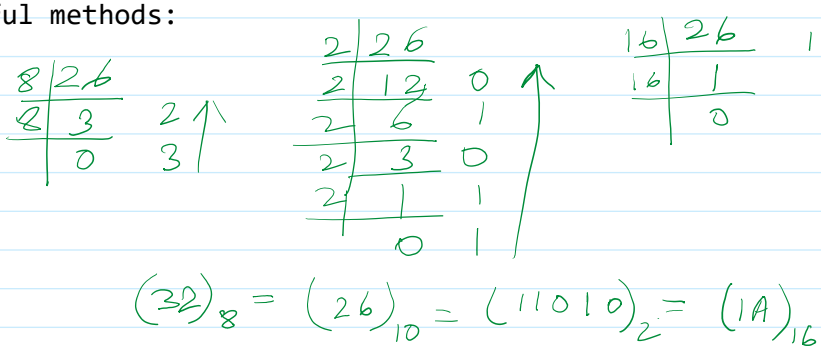
Output:



Converting decimal into binary, octal and hexadecimal

The Integer class provides three useful methods:

- toBinaryString()
- toOctalString()
- toHexString()

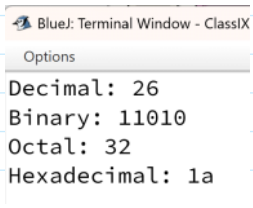



```
package BoxingAndUnBoxing;


/**
 * Write a description of class IntegerConvertsDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```
 */
public class IntegerConvertsDemo
{
    public static void main(String[] args){
        int num = 26;
        System.out.print("\fDecimal: " + num);
        System.out.print("\nBinary: " + Integer.toBinaryString(num));
        System.out.print("\nOctal: " + Integer.toOctalString(num));
        System.out.print("\nHexadecimal: " + Integer.toHexString(num));
    }
}
```

Output:



Finding Integer limits

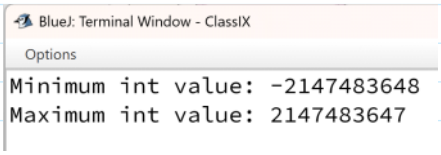
The Integer class provides two constants:

- Integer.MAX_VALUE
- Integer.MIN_VALUE

```
package BoxingAndUnBoxing;


/**
 * Write a description of class IntegerLimitDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class IntegerLimitDemo
{
    public static void main(String[] args){
        System.out.print("\fMinimum int value: " + Integer.MIN_VALUE);
        System.out.print("\nMaximum int value: " + Integer.MAX_VALUE);
    }
}
```

Output:



The int data type occupies 32 bits (4 bytes).

2^31-1 = 2147483647

Character Wrapper Class

The Character class is a rapper class for the primitive care data type.
It provides several useful methods for checking and manipulating characters.

Important methods of Character class

Method	Description
isLetter()	Checks whether a character is an alphabet.
isDigit()	Checks whether a character is a digit.
isLetterOrDigit()	Checks whether a character is alphabet or digit.
isUpperCase()	Checks whether a character is upper case.
isLowerCase()	Checks whether a character is lower case.
toUpperCase()	Convert the character into uppercase.
toLowerCase()	Converts a character into lower case.
isWhiteSpace()	Checks whether a character is white space.
isAlphabetic()	Checks whether a character is alphabetic.
isSpaceChar()	Checks whether a character is a Unicode space character.
getNumericalValue()	Returns the numerical value of a character(Not in ICSE/ISC).